Node.js Backend Development

Lecture 5 — MongoDB, CRUD Operations, Postman, Data Design & Mongoose

Detailed, topic-by-topic notes based on the uploaded lecture transcript

Learning focus: understand the database layer, explore MongoDB locally, perform basic data operations, test API requests with Postman, and understand why Mongoose is commonly used with Node.js.

Contents

- 1. Lecture context and learning approach
- 2. Client–server–database architecture
- 3. Why databases are needed: restaurant example
- 4. SQL and NoSQL; MongoDB terminology
- 5. MongoDB server, shell, and local setup
- 6. Basic MongoDB shell commands
- 7. CRUD operations in MongoDB
- 8. MongoDB Compass GUI
- 9. Data design and API request formats
- 10. GET vs POST and testing with Postman
- 11. How Node.js and MongoDB communicate
- 12. Native MongoDB driver vs Mongoose
- 13. Key terminology, mistakes, and revision checklist
- 14. Practice tasks

1. Lecture context and learning approach

The lecture continues from the previous lesson on Node.js, Express, APIs, JSON, and request/response handling. Its main goal is to introduce MongoDB as the database used in the course, demonstrate basic database operations, show how API data can be sent through Postman, and explain the role of a database driver and Mongoose.

Learning advice from the instructor: follow the concepts in order, avoid letting unrelated old doubts interrupt every new topic, and pause to complete each setup or practical step before continuing. If an error appears, try to identify the exact step that failed and consult official documentation or a targeted tutorial.

2. Client–server–database architecture

A typical web application can be understood as three cooperating parts:

- **Client / frontend:** the website or app the user interacts with. It sends requests to an API and displays the response. Examples mentioned in the lecture include React, Vue, and plain HTML/CSS/JavaScript.
- **Application server:** the Node.js server, commonly built with Express in this course. It receives requests, applies application logic, and decides whether data can be returned directly or must be fetched from the database.
- **Database server:** the database system stores and retrieves persistent application data. MongoDB runs as its own service; it is separate from the Node.js/Express server.

Request flow: client → API endpoint on Node.js/Express → database query when needed → database result → Node.js response → client.

The restaurant analogy used in the lecture: the customer is the client, the waiter is the application server, and the kitchen/chef represents the database side. A waiter can answer a simple request directly, but asks the kitchen when a request needs stored information or work that the waiter cannot perform alone.

Important distinction: Node.js server and MongoDB server are not the same server. They run as separate processes and communicate with each other.

3. Why databases are needed

Applications need to retain information between requests and after a program restarts. A restaurant application might need to store staff records, menu items, prices, ingredients, item categories, and sales counts. A user-based application might store names, email addresses, and other account fields.

Without a database, the application would have no reliable central place to retrieve saved records. The server processes requests; the database provides persistent storage and retrieval.

What a record might contain

- Person: name, designation, phone number, email, address, and salary.
- Menu item: name, price, taste/category, whether it is a drink, ingredients, and number sold.

The exact fields depend on the application's requirements. The lecture's key point is that the data should be planned before the application starts storing it.

4. SQL and NoSQL; MongoDB terminology

The lecture briefly compares SQL databases such as PostgreSQL and Oracle with MongoDB, a NoSQL document database. It recommends understanding SQL as a useful foundational skill, while using MongoDB for the course's practical examples.

MongoDB stores data as BSON documents, which look similar to JSON objects. Instead of a relational table/row/column vocabulary, MongoDB commonly uses database/collection/document/field.

Relational SQL term	MongoDB term	Meaning
Database	Database	A named container for related data
Table	Collection	A group of related records/documents
Row / record	Document	One stored item or entity
Column	Field	One property inside a record
Primary key (concept)	_id	Unique identifier for a document

MongoDB automatically creates an **_id** field when a document is inserted without one. It is normally an ObjectId and uniquely identifies that document within the collection.

5. MongoDB server, shell, and local setup

The lecture asks learners to install MongoDB Community Server for their operating system, then verify the installation and start the database service. Installation steps differ by Windows/macOS/Linux and may change between versions, so use the current official installation instructions for your exact operating system.

Server vs shell

- **MongoDB server (`mongod`):** the database process that stores data and accepts database connections.
- **MongoDB Shell (`mongosh`):** a command-line client used to connect to the server and run database commands.
- **MongoDB Compass:** a graphical application for browsing and editing databases without typing every command.

The instructor demonstrates checking the installed version, starting the database service, opening the shell, listing databases, and exiting the shell. The exact service-start command differs by operating system and installation method.

Typical shell commands

```
mongod --version
```

Checks the server version in installations where this command is available.

```
mongosh
```

Opens the MongoDB shell and connects to the default local instance if it is running and reachable.

```
show dbs
```

Lists databases that have data to show.

```
use myDatabase
```

Switches the shell context to the named database. MongoDB may not show a new database in `show dbs` until data has been stored in it.

```
show collections
```

Lists the collections in the current database.

```
exit
```

Leaves the shell.

Setup troubleshooting: if the shell cannot connect, first check whether MongoDB Server is installed and running, then verify the host/port and any service instructions for your OS. Installing `mongosh` alone does not necessarily install or start the database server.

6. Basic MongoDB shell commands

The transcript demonstrates selecting a database, listing collections, creating a collection, inserting documents, reading documents, filtering, updating, and deleting.

Choose or create a database

```
use practiceDB
```

Switches to `practiceDB`. A database is persisted and becomes visible in database listings after it contains data.

Create a collection

```
db.createCollection("users")
```

Creates a collection called `users`. MongoDB can also create a collection implicitly when data is first inserted into it.

Insert one document

```
db.users.insertOne({ username: "alex", age: 25 })
```

Adds one document to the `users` collection. If `_id` is omitted, MongoDB generates one.

Read documents

```
db.users.find()
```

Returns all documents in the collection.

```
db.users.find({ age: { $gt: 21 } })
```

Filters documents where age is greater than 21. MongoDB comparison operators commonly begin with `$`, such as `$gt` (greater than), `$lt` (less than), `$gte` (greater than or equal), and `$lte` (less than or equal).

```
db.users.find({ username: "alex" })
```

Finds documents whose `username` field equals `alex`.

Update a document

```
db.users.updateOne({ username: "alex" }, { $set: { age: 26 } })
```

The first argument identifies which document to match; the second describes the update. `$set` changes the specified field without replacing the entire document.

Delete a document

```
db.users.deleteOne({ username: "alex" })
```

Deletes one matching document. Always verify the filter before running a deletion.

Note: the transcript uses simplified examples and sometimes refers to commands informally. The examples above use current, conventional `mongosh` syntax so they can be practiced more reliably.

7. CRUD operations in MongoDB

CRUD is the standard shorthand for the four core data operations:

Letter	Operation	Purpose	MongoDB example
C	Create	Insert new data	insertOne(...)
R	Read	Retrieve data	find(...)
U	Update	Change existing data	updateOne(...)
D	Delete	Remove data	deleteOne(...)

The instructor emphasizes that update operations need a filter to identify the target document and an update specification describing what should change. Read operations can return many documents, so use a filter when you need only matching records.

8. MongoDB Compass GUI

MongoDB Compass is a graphical user interface (GUI) for viewing and managing MongoDB data. It can display databases, collections, documents, and fields visually, which becomes more convenient than reading large amounts of output in a terminal.

- Connect to a local MongoDB instance using the connection URI shown by your installation, commonly `mongodb://127.0.0.1:27017` for a default local server.
- Open a database and collection to browse documents.
- Insert a document, edit fields, or delete a document through the interface.
- The transcript also mentions Robo 3T as another GUI option; the instructor prefers Compass in this demonstration.

Compass is a client tool; it does not replace the MongoDB server. The server must be running and reachable for Compass to connect.

9. Data design and API request formats

Before connecting a frontend form to a database, the team needs to decide what information is required and the format in which it will be sent. The lecture's restaurant examples use a person/staff object and a menu-item object.

Example person document

```
{ "name": "Alex", "age": 28, "designation": "chef", "mobile": "0000000000", "email":
"alex@example.com", "address": "Delhi", "salary": 50000 }
```

Example menu-item document

```
{ "name": "Mango smoothie", "price": 4.99, "taste": "sweet", "isDrink": true, "ingredients":
["mango", "milk"], "soldCount": 45 }
```

These are illustrative data shapes based on the lecture's examples, not a schema the course has already implemented. The frontend usually collects values in a form and sends them to an API. The server should validate the received data before saving it.

Data design principle: agree on field names, types, required values, and allowed formats. Do not accept arbitrary unwanted fields just because a client sends them.

10. GET vs POST and testing with Postman

The lecture revisits API methods and introduces Postman as a way to act like a frontend while the real frontend has not yet been built.

GET

- Used to request or retrieve information from the server.
- In the lecture's simple Express example, visiting a route such as `/` or `/chicken` with GET returns a response.
- GET is not intended to create a new database record.

POST

- Used to send data to the server, commonly to create a record.
- The client sends data in the request body, often as JSON.
- The server needs a matching POST route and must parse the request body before it can process the data.

Postman workflow

- Start the Node.js server.
- Enter the API URL, for example `http://localhost:3000/items`.
- Choose the correct HTTP method, such as POST.
- Open Body, choose raw, then choose JSON (if the request is JSON).
- Paste the JSON payload and click Send.
- Inspect the response and server logs. A route that does not exist for the chosen method may return an error such as "Cannot POST /items".

The transcript shows sending person and menu-item JSON through Postman and explaining that a response acknowledging receipt is not proof that the data has been saved. The route must actually implement the database insert operation for persistence to occur.

Minimal Express concept example

```
app.post("/items", (req, res) => {\n console.log(req.body);\n res.json({ message: "Request received" });\n});
```

This is only a conceptual example of receiving a POST request. To read JSON bodies in a standard Express setup, middleware such as `express.json()` must be configured before the route. This example logs and acknowledges data; it does not save it to MongoDB.

11. How Node.js and MongoDB communicate

Node.js and MongoDB speak through a database driver. The driver is a library that provides functions for connecting to MongoDB, issuing database operations, and receiving results/errors in a form Node.js code can use.

Conceptual flow:

```
Express route → JavaScript code → MongoDB driver/library → MongoDB server
```

The driver handles the protocol-level communication between the Node.js process and MongoDB. When the database returns a result, the library exposes it to the application code. It can also surface connection, query, and validation-related errors.

The lecture distinguishes the official MongoDB Node.js driver from Mongoose. Both help Node.js work with MongoDB, but Mongoose adds a higher-level modeling layer.

12. Native MongoDB driver vs Mongoose

Official MongoDB Node.js driver

The native driver is the official package for communicating with MongoDB from Node.js. It provides direct access to collections and operations, but the application developer must implement more of the surrounding structure and validation as needed.

Mongoose

Mongoose is an Object Data Modeling (ODM) library commonly used with MongoDB and Node.js. It adds a model-oriented layer and features such as schemas, model definitions, validation, and convenient query methods.

Feature	Native MongoDB driver	Mongoose
Connection / queries	Direct driver APIs	Uses the driver underneath
Data shape	You manage conventions yourself	Schemas describe expected fields and types
Validation	Implement the validation you need	Schema validators can help enforce rules
Abstraction	Lower-level and direct	Higher-level, model-based API
Typical trade-off	More control, more responsibility	More structure and convenience

Why the instructor prefers Mongoose for the course

- A schema acts like a blueprint for a document, e.g. a person might have `name`, `age`, and `email` fields.
- Schema validation can catch missing or incorrectly typed values before an operation is saved, depending on the configured rules.
- Models provide a consistent place to define how application code works with a collection.
- It can reduce repetitive low-level code and make database interaction more structured.

The instructor says Mongoose is widely used in Node.js projects and recommends it over directly using the native driver for the course. This is a teaching preference, not a claim that the native driver is obsolete; both approaches remain valid.

Important next-step boundary

This lecture explains why the driver/Mongoose layer is needed, but the transcript ends by previewing the next lesson: actually integrating MongoDB with the Node.js server so API-submitted data is stored in the database. Do not assume the POST examples shown here have already been connected to persistent storage.

13. Key terminology, mistakes, and revision checklist

Terms to remember

- **Database:** container for related application data.
- **Collection:** group of MongoDB documents; roughly analogous to a SQL table.
- **Document:** one JSON-like record.
- **Field:** a named property inside a document.
- **ObjectId / _id:** MongoDB's automatically generated identifier when one is not supplied.
- **CRUD:** Create, Read, Update, Delete.
- **MongoDB server:** process that stores and serves database data.
- **mongosh:** command-line shell for issuing MongoDB commands.

- **Compass:** graphical interface for exploring MongoDB.
- **Postman:** tool for sending HTTP requests to test APIs.
- **Driver:** library enabling Node.js to communicate with MongoDB.
- **Mongoose / ODM:** model-based abstraction with schemas and validation support.

Common mistakes

- Confusing the Express server with the MongoDB server. They are separate processes.
- Opening `mongosh` when the MongoDB server is not running or is not reachable.
- Using GET when the route expects POST, or sending POST to an endpoint that has no POST handler.
- Sending malformed JSON or choosing the wrong body type in Postman.
- Thinking that a success message means data was saved, even when the route only returns a response.
- Updating or deleting without a careful filter.
- Assuming every document must have the same shape in raw MongoDB; use explicit application validation or a Mongoose schema when a consistent shape is required.

Self-check

- What is the difference between the Node.js server and MongoDB server?
- Explain database, collection, document, and field using a users example.
- What does MongoDB's _id field do?
- What is the difference between GET and POST?
- How do you send JSON through Postman?
- What do insertOne, find, updateOne, and deleteOne do?
- What does MongoDB Compass provide?
- What is a database driver?
- What is Mongoose, and how is it different from the native driver?
- Why does a POST route need to be connected to a database operation to persist data?

14. Practice tasks

Complete these in order. Keep the examples in a practice database so you do not accidentally modify important data.

- Install MongoDB Community Server and MongoDB Shell for your operating system; verify the server is running.
- Open `mongosh`, create/use `practiceDB`, and create a `users` collection.
- Insert two documents: `db.users.insertOne({ username: "alex", age: 25 })` and `db.users.insertOne({ username: "prince", age: 23 })`.
- Run `db.users.find()` and `db.users.find({ age: { $gt: 21 } })`.
- Update Alex to age 26 with `db.users.updateOne({ username: "alex" }, { $set: { age: 26 } })`.
- Delete only Alex with `db.users.deleteOne({ username: "alex" })`, then verify the remaining documents.
- Open Compass and confirm the collection and remaining document are visible.
- Start the lecture's Express server and use Postman to compare GET and POST behavior on the routes that actually exist in your code.
- Explain in your own words why Mongoose schemas can be helpful before attempting the next lecture's integration steps.

Quick revision sheet

Architecture: Client → Express/Node.js → MongoDB (when data is needed) → Express → Client.

MongoDB vocabulary: Database → Collection → Document → Field.

CRUD: `insertOne()` → `find()` → `updateOne()` → `deleteOne()`.

HTTP: GET retrieves; POST submits data to a route. The route must explicitly implement saving if persistence is intended.

Tools: mongosh = command line; Compass = GUI; Postman = HTTP API tester.

Integration: Node.js communicates with MongoDB through a driver. Mongoose builds a schema/model layer on top of the driver.

Next lecture preview: connect MongoDB/Mongoose to Node.js and make API data persist in the database.